Redes Neuronales (2026)

Ayudas Guía Nº10: Autoencoder con Fashion-MNIST y PyTorch
(Trabajo en progresso...)[*]

Índice

Antes de comenzar: de clasificación a reconstrucción **5**
¿Por qué no basta con aprender la identidad? . . . 5

1 Ejercicio 1: Preparar el entorno **5**
1.1 Solución completa . . . 5
1.2 Explicación de cada importación . . . 6
1.2.1 `torch` . . . 6
1.2.2 `torch.nn` . . . 6
1.2.3 `DataLoader` y `random_split` . . . 6
1.2.4 `datasets` y `transforms` . . . 6
1.2.5 `matplotlib`, `numpy` y `random` . . . 6
1.3 Semillas y dispositivo . . . 7
1.3.1 ¿Qué es una semilla? . . . 7
1.3.2 CPU y GPU . . . 7

2 Ejercicio 2: Cargar y preparar Fashion-MNIST **7**
2.1 Transformación: por qué usamos solamente `ToTensor()` . . . 7
2.2 Train, validation y test . . . 8
2.2.1 El generador local para el split . . . 8
2.3 Inspeccionar un ejemplo . . . 8
2.4 Mosaico 3×3 con etiquetas . . . 9
2.4.1 ¿Por qué un generador separado para la visualización? . . . 9
2.4.2 ¿Qué hace `squeeze()`? . . . 9

3 Ejercicio 3: Crear los DataLoaders **9**
3.1 ¿Qué es un mini-batch? . . . 10
3.2 ¿Por qué `shuffle=True` sólo para train? . . . 10

4 Ejercicio 4: Comprender el autoencoder **10**
4.1 Modelo matemático para una sola imagen . . . 10
4.2 Representación conceptual por mini-batch . . . 11
4.3 Bottleneck y capacidad . . . 11

5 Ejercicio 5: Definir el autoencoder en PyTorch **12**
5.1 Solución completa . . . 12
5.2 `nn.Module` . . . 12
5.3 Encoder . . . 12
5.3.1 `nn.Flatten()` . . . 12
5.3.2 `nn.Linear(784, latent_dim)` . . . 12
5.3.3 ReLU . . . 12
5.3.4 Dropout . . . 13

[*]Reportar errores a: `tristan.osan@unc.edu.ar`

5.4 Decoder . . . 13
5.4.1 nn.Linear(latent_dim, 784) . . . 13
5.4.2 Sigmoid . . . 13
5.5 Recuperar la forma de una imagen . . . 13
5.6 Comprobar dimensiones . . . 14
5.6.1 ¿Por qué eval() en esta comprobación? . . . 14
5.6.2 ¿Por qué torch.no_grad()? . . . 14

6 Ejercicio 6: Función de pérdida MSE 14
6.1 Interpretación . . . 14
6.2 ¿Qué promedia PyTorch en un mini-batch? . . . 15

7 Ejercicio 7: Elegir el optimizador 15
7.1 SGD . . . 15
7.2 Adam . . . 15

8 Ejercicio 8: Implementar una época de entrenamiento 15
8.1 Solución completa . . . 16
8.2 Explicación paso a paso . . . 16
8.2.1 model.train() . . . 16
8.2.2 for images, _ in data_loader . . . 16
8.2.3 Mover el mini-batch al dispositivo . . . 16
8.2.4 Borrar gradientes anteriores . . . 16
8.2.5 Forward . . . 16
8.2.6 Pérdida . . . 16
8.2.7 Backward . . . 17
8.2.8 Actualizar parámetros . . . 17

9 Ejercicio 9: Implementar la evaluación 17
9.1 Solución completa . . . 17
9.2 @torch.no_grad() . . . 17
9.3 model.eval() . . . 17
9.4 Acumulación correcta de la pérdida . . . 18
9.4.1 Sutileza específica de MSE . . . 18

10 Ejercicio 10: Crear el modelo y el optimizador 18
10.1 ¿Por qué volver a fijar la semilla? . . . 18
10.2 model.parameters() . . . 18
10.3 Reconstrucciones antes del entrenamiento . . . 19
10.3.1 ¿Qué deberíamos esperar visualmente? . . . 19
10.3.2 ¿Por qué se usan imágenes de validation? . . . 19
10.3.3 ¿Por qué model.eval()? . . . 19

11 Ejercicio 11: Entrenar el modelo 19
11.1 ¿Qué significa una época? . . . 20
11.2 Una distinción pedagógica importante: loss durante entrenamiento vs loss al final de la época . . . 20
11.3 ¿Importa que train_loader tenga shuffle=True al evaluar? . . . 20

12 Ejercicio 12: Graficar las pérdidas 20
12.1 Cómo leer las curvas . . . 21

13 Ejercicio 13: Comparar SGD y Adam usando validation 21
13.1 Procedimiento correcto . . . 21
13.2 Por qué test no se usa para elegir . . . 21
13.3 ¿Qué significa “comparación justa”? . . . 21

14 Ejercicio 14: Evaluación final sobre test **22**
14.1 Qué no debemos hacer después . . . 22

15 Ejercicio 15: Visualizar reconstrucciones sobre test **22**
15.1 Qué observar . . . 22

16 Ejercicio 16: Observar el espacio latente **23**
16.1 ¿Qué contiene el código latente? . . . 23
16.2 ReLU y signo de las componentes . . . 23
16.3 ¿Por qué Dropout no altera estos códigos en esta celda? . . . 23

17 Ejercicio 17: Preguntas conceptuales - soluciones completas **23**
17.1 ¿Por qué las etiquetas de Fashion-MNIST no son necesarias? . . . 23
17.2 ¿Qué representa el código latente? . . . 23
17.3 ¿Qué significa el bottleneck? . . . 23
17.4 ¿Por qué usamos MSELoss? . . . 24
17.5 ¿Qué hace `optimizer.zero_grad()`? . . . 24
17.6 ¿Qué hace `loss.backward()`? . . . 24
17.7 ¿Qué hace `optimizer.step()`? . . . 24
17.8 ¿Por qué volvemos a evaluar train después de cada época? . . . 24
17.9 ¿Qué diferencia hay entre `model.train()` y `model.eval()`? . . . 24
17.10 ¿Qué hace `Dropout(p)` durante `model.train()`? . . . 24
17.11 ¿Qué ocurre con dropout durante `model.eval()`? . . . 24
17.12 ¿Qué significa elegir `DROPOUT_P=0.0`? . . . 24
17.13 ¿Por qué no colocamos dropout en la capa final? . . . 25
17.14 ¿Por qué SGD y Adam usan learning rates diferentes? . . . 25

18 Ejercicio 18: Experimentos sugeridos - guía de solución **25**
18.1 Experimento A: dimensión latente . . . 25
18.1.1 Efectos esperados . . . 25
18.1.2 Cómo resolver el experimento correctamente . . . 26
18.2 Experimento B: dropout . . . 26
18.2.1 Interpretación esperada . . . 26
18.3 Experimento C: número de épocas . . . 26
18.4 Experimento D: SGD versus Adam . . . 26

19 Errores frecuentes y cómo explicarlos **27**
19.1 Usar las etiquetas en la loss . . . 27
19.2 Olvidar mover las imágenes al dispositivo . . . 27
19.3 Olvidar `model.eval()` durante evaluación . . . 27
19.4 Creer que `no_grad()` desactiva dropout . . . 27
19.5 Usar test para seleccionar el optimizador . . . 27
19.6 Confundir vector latente con etiqueta . . . 27
19.7 Interpretar dimensión latente como bytes comprimidos . . . 27

20 Resumen conceptual **27**

A Código completo consolidado **28**

B Referencias mínimas **35**

Objetivos generales

- explicar la diferencia entre clasificación y reconstrucción;
- describir la estructura encoder–código latente–decoder;
- comprender el concepto de *bottleneck*;
- interpretar una imagen de Fashion-MNIST como tensor de forma $(1, 28, 28)$;
- construir un autoencoder totalmente conectado ("*fully-connected*") con PyTorch;
- comprender el papel de `Flatten`, `Linear`, ReLU, `Dropout` y Sigmoid;
- explicar por qué la salida del modelo debe tener la misma forma que la entrada;
- interpretar el error cuadrático medio (MSE) como pérdida de reconstrucción;
- distinguir el **modo de entrenamiento** de la red del **modo de evaluación**;
- explicar qué hacen `zero_grad()`, `backward()` y `step()`;
- comparar SGD y Adam usando *validation*, sin utilizar *test* para tomar decisiones;
- interpretar curvas de *train loss* y *validation loss*;
- visualizar reconstrucciones antes y después del entrenamiento;
- interpretar la dimensión del código latente y discutir el efecto de modificarla.

Mapa general de la guía

El flujo conceptual puede resumirse así:

$$\boxed{\text{imagen } x} \longrightarrow \boxed{\text{encoder}} \longrightarrow \boxed{\text{código latente } z} \longrightarrow \boxed{\text{decoder}} \longrightarrow \boxed{\text{reconstrucción } \hat{x}}$$

El objetivo es lograr

$$\hat{x} \approx x.$$

Durante entrenamiento:

$$\boxed{\text{mini-batch}} \rightarrow \boxed{\text{forward}} \rightarrow \boxed{\text{MSE}} \rightarrow \boxed{\text{backward}} \rightarrow \boxed{\text{optimizer.step()}}$$

Al finalizar cada época:

$$\boxed{\text{evaluar train}} \rightarrow \boxed{\text{evaluar validation}} \rightarrow \boxed{\text{guardar pérdidas}}$$

Finalmente:

$$\boxed{\text{comparar configuraciones con validation}} \rightarrow \boxed{\text{elegir configuración}} \rightarrow \boxed{\text{test final}}$$

Idea clave: Un autoencoder no intenta adivinar una etiqueta. Intenta reconstruir su propia entrada. En esta guía, las etiquetas de Fashion-MNIST son útiles para explorar visualmente los datos, pero no intervienen en la función de pérdida.

Antes de comenzar: de clasificación a reconstrucción

En un problema de clasificación supervisada trabajamos con pares

$$(x_i, y_i),$$

donde x_i es una imagen y y_i es su clase. Una red de clasificación aprende una función

$$f_\theta(x_i) \approx y_i.$$

En un *autoencoder*, en cambio, queremos que el objetivo sea reconstruir la propia entrada:

$$f_\theta(x_i) \approx x_i.$$

Por eso, el problema de aprendizaje puede escribirse como

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), x_i),$$

donde la función de error ℓ será, en esta guía, el error cuadrático medio (MSE).

¿Por qué no basta con aprender la identidad?

A primera vista podría parecer trivial pedir que la red devuelva lo mismo que recibe. La parte interesante aparece porque imponemos una representación intermedia mucho más pequeña que la entrada.

Una imagen tiene

$$28 \times 28 = 784$$

píxeles. En la configuración base utilizaremos un código latente de dimensión

$$64.$$

Por lo tanto, la red debe hacer aproximadamente

$$784 \to 64 \to 784.$$

La capa de 64 componentes funciona como un *bottleneck*: obliga a la red a representar información útil de la imagen usando menos variables internas.

Observación: Es importante aclarar que “64 en lugar de 784” es una compresión de representación, no necesariamente una compresión de archivo. Los 64 valores son números de punto flotante y el autoencoder no está diseñado aquí como algoritmo de compresión JPEG.

1. Ejercicio 1: Preparar el entorno

1.1. Solución completa

```
import random

import matplotlib.pyplot as plt
import numpy as np
import torch

from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
```

1.2. Explicación de cada importación

1.2.1. `torch`

PyTorch es la biblioteca central. Proporciona:

- tensores multidimensionales;
- operaciones sobre CPU y GPU;
- diferenciación automática;
- módulos para construir redes neuronales;
- optimizadores.

1.2.2. `torch.nn`

El módulo `nn` contiene las piezas con las que construiremos el autoencoder:

- `nn.Module`: clase base de todo modelo;
- `nn.Sequential`: secuencia de módulos;
- `nn.Flatten`: aplana cada imagen;
- `nn.Linear`: transformación afín totalmente conectada;
- `nn.ReLU`: no linealidad;
- `nn.Dropout`: regularización;
- `nn.Sigmoid`: salida acotada entre 0 y 1;
- `nn.MSELoss`: error cuadrático medio.

1.2.3. `DataLoader` y `random_split`

`DataLoader` permite recorrer un *dataset* en lotes. `random_split` permite dividir el conjunto de 60.000 ejemplos oficiales de entrenamiento en *train* y *validation*.

1.2.4. `datasets` y `transforms`

`datasets.FashionMNIST` descarga y expone el *dataset*. `transforms.ToTensor()` transforma las imágenes a tensores numéricos.

1.2.5. `matplotlib`, `numpy` y `random`

`matplotlib` se utiliza para mosaicos y curvas. `numpy` y `random` se utilizan aquí, además de sus usos generales, para controlar fuentes de aleatoriedad.

1.3. Semillas y dispositivo

```
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("PyTorch:", torch.__version__)
print("Dispositivo:", device)
```

1.3.1. ¿Qué es una semilla?

Un generador pseudoaleatorio produce una secuencia determinista a partir de un estado inicial. La semilla fija ese estado. Por eso, bajo condiciones equivalentes, usar la misma semilla ayuda a repetir:

- inicializaciones de pesos;
- divisiones train/validation;
- órdenes aleatorios de ejemplos;
- patrones de dropout.

Idea clave: La semilla mejora la reproducibilidad, pero no garantiza igualdad bit a bit entre cualquier GPU, versión de biblioteca y plataforma.

1.3.2. CPU y GPU

La expresión

```
torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

selecciona GPU cuando CUDA está disponible; de lo contrario usa CPU. Más adelante el modelo y los mini-batches deberán encontrarse en el mismo dispositivo.

2. Ejercicio 2: Cargar y preparar Fashion-MNIST

2.1. Transformación: por qué usamos solamente `ToTensor()`

```
transform = transforms.ToTensor()
```

`ToTensor()` realiza dos cambios fundamentales:

1. convierte la imagen en un `torch.Tensor`;
2. transforma los valores de cada píxel (0-255) aproximadamente al intervalo $[0, 1]$.

Cada imagen queda con forma

$$(1, 28, 28),$$

donde el primer 1 corresponde al número de canales (un solo canal, en este caso).

2.2. Train, validation y test

```
full_train_set = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=transform
)

test_set = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=transform
)

split_generator = torch.Generator().manual_seed(SEED)

train_set, valid_set = random_split(
    full_train_set,
    [50_000, 10_000],
    generator=split_generator
)
```

Fashion-MNIST contiene oficialmente:

- 60.000 ejemplos de entrenamiento;
- 10.000 ejemplos de test.

La guía transforma los 60.000 ejemplos iniciales en:

Conjunto	Cantidad	Función
Train	50.000	actualizar parámetros
Validation	10.000	comparar configuraciones
Test	10.000	evaluación final

2.2.1. El generador local para el split

```
split_generator = torch.Generator().manual_seed(SEED)
```

crea un generador pseudoaleatorio independiente. Al pasarlo a `random_split`, la división train/validation puede repetirse sin depender directamente del estado global del generador de PyTorch.

2.3. Inspeccionar un ejemplo

```
image, label = train_set[0]

print("Shape:", image.shape)
print("Etiqueta:", label)
print("Minimo:", image.min().item())
print("Maximo:", image.max().item())
```

Fashion-MNIST sigue entregando un par

$$(\text{image}, \text{label}).$$

En esta guía no modificamos el dataset. Esa decisión hace que el código sea más simple. Durante entrenamiento simplemente ignoraremos `label`.

Idea clave: La etiqueta sigue existiendo porque Fashion-MNIST fue diseñado para clasificación. El autoencoder no la necesita para aprender.

2.4. Mosaico 3×3 con etiquetas

```
class_names = {
    0: "T-shirt/top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle boot",
}

display_generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(
    len(train_set),
    generator=display_generator
)[:9].tolist()

fig, axes = plt.subplots(3, 3, figsize=(7, 7))

for ax, idx in zip(axes.flat, indices):
    image, label = train_set[idx]
    ax.imshow(image.squeeze(), cmap="gray")
    ax.set_title(class_names[label])
    ax.axis("off")

plt.tight_layout()
plt.show()
```

2.4.1. ¿Por qué un generador separado para la visualización?

El objeto `display_generator` permite elegir siempre los mismos nueve índices sin consumir el estado pseudoaleatorio global. Esto evita que una acción puramente visual modifique accidentalmente la secuencia de números aleatorios que luego podría utilizar la red.

2.4.2. ¿Qué hace `squeeze()`?

La imagen tiene forma $(1, 28, 28)$. Para `imshow`, es conveniente una matriz $(28, 28)$. En este caso `squeeze()` elimina la dimensión de tamaño 1.

Observación: Conviene aprovechar este mosaico para señalar que las etiquetas se muestran por razones interpretativas. No se debe confundir “tener etiquetas disponibles” con “usar etiquetas en la función de pérdida (*loss*)”.

3. Ejercicio 3: Crear los DataLoaders

```
BATCH_SIZE = 100 #Tamaño del mini-batch

train_loader = DataLoader(
    train_set,
    batch_size=BATCH_SIZE,
```

```
    shuffle=True
)

valid_loader = DataLoader(
    valid_set,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_set,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

3.1. ¿Qué es un mini-batch?

Si $B = 100$, un mini-batch contiene 100 imágenes. La forma de las imágenes pasa de

$$(1, 28, 28)$$

a

$$(100, 1, 28, 28).$$

La primera dimensión es la dimensión del *mini-batch*.

```
images, labels = next(iter(train_loader))

print("images.shape:", images.shape)
print("labels.shape:", labels.shape)
```

produce conceptualmente:

- `images.shape = (100,1,28,28)`;
- `labels.shape = (100,)`.

3.2. ¿Por qué `shuffle=True` sólo para train?

Durante entrenamiento es habitual cambiar el orden de presentación de los ejemplos. En validation y test no se actualizan parámetros; por eso no necesitamos mezclar los datos.

Idea clave: `shuffle=True` cambia el orden, no cambia qué ejemplos pertenecen a *train*.

4. Ejercicio 4: Comprender el autoencoder

4.1. Modelo matemático para una sola imagen

Conceptualmente podemos aplanar una imagen y representarla como un **vector columna**

$$\boldsymbol{x} \in \mathbb{R}^{784\times 1}.$$

El encoder lineal utiliza una matriz

$$W_{\text{enc}} \in \mathbb{R}^{64\times 784}$$

y un vector de bias

$$\boldsymbol{b}_{\text{enc}} \in \mathbb{R}^{64\times 1}.$$

Antes de la activación:

$$\boldsymbol{a} = W_{\text{enc}}\boldsymbol{x} + \boldsymbol{b}_{\text{enc}}.$$

Después de ReLU:

$$\boldsymbol{z} = \text{ReLU}(\boldsymbol{a}),$$

con

$$\boldsymbol{z} \in \mathbb{R}^{64\times 1}.$$

El decoder utiliza

$$W_{\text{dec}} \in \mathbb{R}^{784\times 64}, \qquad \boldsymbol{b}_{\text{dec}} \in \mathbb{R}^{784\times 1},$$

y produce

$$\hat{\boldsymbol{x}} = \sigma\left(W_{\text{dec}}\boldsymbol{z} + \boldsymbol{b}_{\text{dec}}\right).$$

Por lo tanto:

$$784 \to 64 \to 784.$$

4.2. Representación conceptual por mini-batch

Si conceptualizamos un mini-batch colocando un ejemplo por columna,

$$X = \left[\boldsymbol{x}^{(1)}\ \boldsymbol{x}^{(2)}\ \cdots\ \boldsymbol{x}^{(B)}\right] \in \mathbb{R}^{784\times B}.$$

Sin embargo, PyTorch utiliza convención *batch-major*; después de `Flatten`, el tensor tiene forma

$$(B, 784).$$

No hay contradicción: es una diferencia de convención de almacenamiento e implementación.

4.3. Bottleneck y capacidad

Con `LATENT_DIM=64`, la razón entre dimensiones es

$$\frac{64}{784} \approx 0{,}0816.$$

La representación latente tiene alrededor del $8{,}2\,\%$ de la cantidad de componentes de la entrada.

> **Para pensar:** ¿Qué esperamos si usamos 16 componentes? ¿Y si usamos 128? Una dimensión menor fuerza mayor compresión, pero también puede dificultar la reconstrucción.

5. Ejercicio 5: Definir el autoencoder en PyTorch

5.1. Solución completa

```
class Autoencoder(nn.Module):
    def __init__(self, latent_dim=64, p_dropout=0.0):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, latent_dim),
            nn.ReLU(),
            nn.Dropout(p_dropout)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 28 * 28),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        x_hat = self.decoder(z)

        x_hat = x_hat.view(x.size(0), 1, 28, 28)

        return x_hat
```

5.2. `nn.Module`

Todo modelo de PyTorch hereda de `nn.Module`. El método `__init__()` declara las capas y `forward()` describe cómo fluye un tensor a través del modelo.

5.3. Encoder

5.3.1. `nn.Flatten()`

Transforma cada imagen

$$(1, 28, 28)$$

en un vector de 784 componentes. Para un mini-batch:

$$(B, 1, 28, 28) \to (B, 784).$$

5.3.2. `nn.Linear(784, latent_dim)`

Implementa una transformación afín. Si `latent_dim=64`, cada ejemplo pasa de 784 a 64 valores. La cantidad de parámetros de esta capa es

$$784 \times 64 + 64 = 50{,}240.$$

5.3.3. ReLU

$$\mathrm{ReLU}(a) = \mathrm{máx}(0, a).$$

Introduce no linealidad. Sin una activación no lineal, un autoencoder con capas lineales tendría propiedades mucho más restrictivas.

5.3.4. Dropout

`nn.Dropout(p)` actúa durante `model.train()`. Cada activación se anula aleatoriamente con probabilidad p.

Para $p = 0{,}2$:

- cada componente tiene probabilidad $0{,}2$ de ser anulada;
- no significa que exactamente el $20\,\%$ se anule en cada mini-batch;
- PyTorch reescala las activaciones supervivientes por $1/(1-p)$ durante entrenamiento;
- durante `model.eval()`, dropout se comporta como la identidad.

Si

$$p = 0,$$

Dropout queda efectivamente desactivado.

> **Observación:** En este modelo Dropout se coloca en el código latente, no en la salida. Si anuláramos píxeles reconstruidos justo antes de comparar con el objetivo, estaríamos introduciendo ruido directamente en la salida que queremos evaluar.

5.4. Decoder

5.4.1. `nn.Linear(latent_dim, 784)`

Expande el código latente nuevamente a 784 valores. Con dimensión latente 64, esta capa contiene

$$64 \times 784 + 784 = 50{,}960$$

parámetros.

El modelo base tiene entonces

$$50{,}240 + 50{,}960 = 101{,}200$$

parámetros entrenables.

5.4.2. Sigmoid

La Sigmoid aplica

$$\sigma(a) = \frac{1}{1 + e^{-a}}.$$

Su salida queda entre 0 y 1, coherente con `ToTensor()`.

5.5. Recuperar la forma de una imagen

El decoder produce inicialmente un tensor de forma $(B, 784)$. La línea

```
x_hat = x_hat.view(x.size(0), 1, 28, 28)
```

lo transforma en

$$(B, 1, 28, 28).$$

Esto permite comparar directamente `x_hat` con `x` y visualizar la reconstrucción como una imagen.

5.6. Comprobar dimensiones

```
LATENT_DIM = 64
DROPOUT_P = 0.2

torch.manual_seed(SEED)

model_example = Autoencoder(
    latent_dim=LATENT_DIM,
    p_dropout=DROPOUT_P
)

model_example.eval()

with torch.no_grad():
    reconstructions = model_example(images)
    latent_codes = model_example.encoder(images)
```

Se espera:

$$(100, 1, 28, 28) \to (100, 64) \to (100, 1, 28, 28).$$

5.6.1. ¿Por qué `eval()` en esta comprobación?

Porque queremos comprobar dimensiones con un comportamiento determinista. Si el modelo estuviera en modo entrenamiento y $p > 0$, Dropout introduciría una máscara aleatoria. La forma no cambiaría, pero el contenido sí.

5.6.2. ¿Por qué `torch.no_grad()`?

No queremos entrenar. No necesitamos construir el grafo de autograd. Esto reduce memoria y deja claro que estamos haciendo inferencia.

6. Ejercicio 6: Función de pérdida MSE

```
loss_fn = nn.MSELoss()
```

Para una imagen aplanada con $D = 784$ píxeles:

$$\mathrm{MSE}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \frac{1}{D} \sum_{j=1}^{D} (x_j - \hat{x}_j)^2.$$

6.1. Interpretación

Cada término

$$(x_j - \hat{x}_j)^2$$

mide el error cuadrático de un píxel. Elevar al cuadrado:

- evita cancelación entre errores positivos y negativos;
- penaliza más los errores grandes que los pequeños.

Como entrada y salida están aproximadamente en $[0, 1]$, cada error cuadrático de píxel está entre 0 y 1.

6.2. ¿Qué promedia PyTorch en un mini-batch?

Por defecto, `nn.MSELoss(reduction="mean")` promedia sobre **todos los elementos** del tensor. Para un mini-batch de forma $(B, 1, 28, 28)$:

$$L_{\text{mini-batch}} = \frac{1}{B \cdot 784} \sum_{i=1}^{B} \sum_{j=1}^{784} \left(x_j^{(i)} - \hat{x}_j^{(i)} \right)^2 .$$

Como todas las imágenes tienen la misma cantidad de píxeles, también puede interpretarse como el promedio de los MSE individuales de las imágenes del mini-batch.

> **Idea clave:** No utilizamos *accuracy* porque no hay una clase correcta/incorrecta. La métrica principal es el error de reconstrucción.

7. Ejercicio 7: Elegir el optimizador

```
OPTIMIZER_NAME = "Adam"   # Opciones: "SGD" o "Adam"

SGD_LR = 0.01
ADAM_LR = 0.001
```

7.1. SGD

En su forma básica, descenso por gradiente estocástico actualiza los parámetros θ mediante

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

donde η es el learning rate.

En la guía:

$$\eta_{\text{SGD}} = 0{,}01.$$

7.2. Adam

Adam mantiene estadísticas adaptativas de primer y segundo orden de los gradientes. No es necesario desarrollar toda su derivación en esta primera guía, pero sí debe quedar claro que su escala efectiva de actualización es distinta a la de SGD.

En la guía:

$$\eta_{\text{Adam}} = 0{,}001.$$

> **Idea clave:** No existe ninguna regla que obligue a utilizar el mismo learning rate para optimizadores distintos. Comparar SGD y Adam con un único valor de η puede ser una comparación injusta.

8. Ejercicio 8: Implementar una época de entrenamiento

La solución que sigue conserva exactamente el siguiente orden conceptual: modo de entrenamiento, transferencia al dispositivo, limpieza de gradientes, forward, pérdida, backward y actualización.

8.1. Solución completa

```
def train_one_epoch(model, data_loader, loss_fn, optimizer, device):
    model.train()

    for images, _ in data_loader:
        images = images.to(device)

        optimizer.zero_grad()

        reconstructions = model(images)
        loss = loss_fn(reconstructions, images)

        loss.backward()
        optimizer.step()
```

8.2. Explicación paso a paso

8.2.1. `model.train()`

Activa el modo entrenamiento. Para este modelo, el efecto importante es que Dropout pasa a estar activo.

8.2.2. `for images, _ in data_loader`

Fashion-MNIST devuelve imagen y etiqueta. El guión bajo indica que el segundo valor existe, pero decidimos no utilizarlo.

Conceptualmente:

$$\text{input} = x, \qquad \text{target} = x.$$

8.2.3. Mover el mini-batch al dispositivo

```
images = images.to(device)
```

Si el modelo está en GPU y el tensor queda en CPU, PyTorch genera un error. Ambos deben estar en el mismo dispositivo.

8.2.4. Borrar gradientes anteriores

```
optimizer.zero_grad()
```

PyTorch acumula gradientes por defecto. Para que cada mini-batch calcule su propia contribución, se borran los gradientes existentes antes de `backward()`.

8.2.5. Forward

```
reconstructions = model(images)
```

Ejecuta `forward()` para todo el mini-batch.

8.2.6. Pérdida

```
loss = loss_fn(reconstructions, images)
```

Obsérvese que no aparece `labels`. El target es `images`.

8.2.7. Backward

```
loss.backward()
```

Autograd aplica la regla de la cadena y calcula

$$\frac{\partial L}{\partial \theta}$$

para cada parámetro entrenable.

8.2.8. Actualizar parámetros

```
optimizer.step()
```

Esta es la operación que efectivamente modifica pesos y biases.

Observación: Conviene insistir en la separación: `backward()` calcula gradientes; `step()` usa esos gradientes para actualizar parámetros.

9. Ejercicio 9: Implementar la evaluación

9.1. Solución completa

```
@torch.no_grad()
def evaluate(model, data_loader, loss_fn, device):
    model.eval()

    total_loss = 0.0
    total_examples = 0

    for images, _ in data_loader:
        images = images.to(device)

        reconstructions = model(images)
        loss = loss_fn(reconstructions, images)

        batch_size = images.size(0)

        total_loss += loss.item() * batch_size
        total_examples += batch_size

    mean_loss = total_loss / total_examples

    return mean_loss
```

9.2. `@torch.no_grad()`

Desactiva el seguimiento de gradientes dentro de toda la función. Evaluar no requiere backward.

9.3. `model.eval()`

Cambia el comportamiento de módulos que dependen del modo. En nuestro caso, desactiva Dropout.

Idea clave: `no_grad()` y `eval()` resuelven problemas diferentes. El primero afecta autograd; el segundo afecta el comportamiento del modelo. En evaluación usamos ambos.

9.4. Acumulación correcta de la pérdida

La loss de cada mini-batch es un promedio. Si el mini-batch tiene B_b ejemplos y pérdida media L_b, la línea

```
total_loss += loss.item() * batch_size
```

acumula una cantidad proporcional al número de ejemplos.
Al finalizar:

$$L_{\text{dataset}} = \frac{\sum_b B_b L_b}{\sum_b B_b}.$$

Esto es especialmente importante cuando el último mini-batch es más pequeño.

9.4.1. Sutileza específica de MSE

Como `MSELoss` promedia sobre todos los píxeles y todas las imágenes del mini-batch, multiplicar por el número de imágenes es correcto aquí porque todas las imágenes tienen exactamente 784 píxeles. Si los ejemplos tuvieran distinta cantidad de elementos, habría que revisar el esquema de ponderación.

10. Ejercicio 10: Crear el modelo y el optimizador

```
torch.manual_seed(SEED)

model = Autoencoder(
    latent_dim=LATENT_DIM,
    p_dropout=DROPOUT_P
).to(device)

if OPTIMIZER_NAME == "SGD":
    optimizer = torch.optim.SGD(
        model.parameters(),
        lr=SGD_LR
    )

elif OPTIMIZER_NAME == "Adam":
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=ADAM_LR
    )

else:
    raise ValueError("OPTIMIZER_NAME debe ser 'SGD' o 'Adam'.")
```

10.1. ¿Por qué volver a fijar la semilla?

Queremos que la creación del modelo a entrenar parta de una inicialización controlada. Es especialmente útil cuando se compara SGD contra Adam: si el resto del procedimiento se mantiene igual, ambos pueden comenzar desde los mismos parámetros iniciales.

10.2. `model.parameters()`

El optimizador recibe una colección de todos los parámetros entrenables del modelo. No necesitamos listar manualmente cada matriz de pesos.

10.3. Reconstrucciones antes del entrenamiento

La notebook incluye deliberadamente una visualización antes de ejecutar la primera época:

```
@torch.no_grad()
def show_untrained_reconstructions(model, data_loader, device, n=6):
    model.eval()

    images, labels = next(iter(data_loader))
    images = images[:n].to(device)
    labels = labels[:n]

    reconstructions = model(images)
    ...
```

La fila superior muestra imágenes reales; la inferior, reconstrucciones de un modelo con parámetros iniciales aleatorios.

10.3.1. ¿Qué deberíamos esperar visualmente?

Antes del entrenamiento no existe ninguna razón para que la red reconstruya prendas. Como la última activación es Sigmoid y los pesos iniciales suelen producir valores de preactivación cercanos a cero, es frecuente observar salidas grisáceas o patrones sin significado. No es necesario que todas las ejecuciones se vean iguales.

10.3.2. ¿Por qué se usan imágenes de validation?

Porque todavía estamos desarrollando y comprendiendo el modelo. El conjunto de test se reserva para el final.

10.3.3. ¿Por qué `model.eval()`?

Queremos que la figura represente una salida determinista del modelo no entrenado. Si Dropout estuviera activo, dos forward passes podrían producir resultados diferentes.

11. Ejercicio 11: Entrenar el modelo

```
EPOCHS = 10

history = {
    "train_loss": [],
    "valid_loss": []
}

for epoch in range(1, EPOCHS + 1):
    train_one_epoch(
        model,
        train_loader,
        loss_fn,
        optimizer,
        device
    )

    train_loss = evaluate(
        model,
        train_loader,
        loss_fn,
        device
```

```
    )

    valid_loss = evaluate(
        model,
        valid_loader,
        loss_fn,
        device
    )

    history["train_loss"].append(train_loss)
    history["valid_loss"].append(valid_loss)
```

11.1. ¿Qué significa una época?

Una época es un recorrido completo del conjunto de entrenamiento. Con 50.000 ejemplos y mini-batch size 100 hay aproximadamente

$$\frac{50{,}000}{100} = 500$$

mini-batches por época.

Por lo tanto, diez épocas implican aproximadamente 5.000 actualizaciones del optimizador.

11.2. Una distinción pedagógica importante: loss durante entrenamiento vs loss al final de la época

Durante `train_one_epoch()`, los parámetros cambian después de cada mini-batch. Una pérdida registrada allí mezclaría estados distintos del modelo.

En esta guía, después de terminar la época, llamamos a

```
evaluate(model, train_loader, ...)
```

para medir train loss con los **pesos finales de la época**. Luego medimos validation con exactamente los mismos pesos.

Así, las dos cantidades son comparables:

$$L_{\text{train}}^{(e)} \quad \text{y} \quad L_{\text{valid}}^{(e)}$$

corresponden al mismo $\theta^{(e)}$.

11.3. ¿Importa que `train_loader` tenga `shuffle=True` al evaluar?

No para una métrica agregada como esta, siempre que recorramos el dataset completo una vez. Cambiar el orden no cambia el promedio final. Lo importante es que `evaluate()` ejecuta `model.eval()` y no actualiza parámetros.

12. Ejercicio 12: Graficar las pérdidas

```
epochs = range(1, EPOCHS + 1)

plt.figure(figsize=(7, 4))
plt.plot(epochs, history["train_loss"], marker="o", label="train")
plt.plot(epochs, history["valid_loss"], marker="o", label="validation")
plt.xlabel("Epoca")
plt.ylabel("MSE")
plt.title(f"Autoencoder - {OPTIMIZER_NAME}")
plt.legend()
plt.grid(alpha=0.3)
```

```
plt.show()
```

12.1. Cómo leer las curvas

- Si train y validation bajan: el modelo está aprendiendo representaciones útiles.
- Si ambas quedan altas y casi no bajan: puede haber subajuste, learning rate inadecuado o entrenamiento insuficiente.
- Si train sigue bajando pero validation empieza a subir: aparece una señal compatible con sobreajuste.
- Si las curvas son muy inestables: el learning rate puede ser demasiado grande o la optimización demasiado ruidosa.

  Observación: En un autoencoder la inspección visual complementa el MSE. Dos modelos con pérdidas cercanas pueden producir reconstrucciones perceptualmente diferentes.

13. Ejercicio 13: Comparar SGD y Adam usando validation

Esta sección introduce una regla metodológica fundamental: no debemos elegir un hiperparámetro mirando test.

13.1. Procedimiento correcto

1. Entrenar con SGD, manteniendo fijos `LATENT_DIM`, `DROPOUT_P`, `BATCH_SIZE` y `EPOCHS`.
2. Registrar train loss y validation loss.
3. Crear nuevamente el modelo desde cero.
4. Entrenar con Adam, manteniendo fijos los mismos hiperparámetros.
5. Comparar las curvas y la validation loss.
6. Elegir una configuración.
7. Recién entonces evaluar esa configuración en test.

13.2. Por qué test no se usa para elegir

Si evaluáramos muchos optimizadores en test y eligiéramos el que da menor test MSE, test se convertiría de hecho en un nuevo validation set. La estimación final dejaría de ser independiente.

La división conceptual es:

$$\text{train} \rightarrow \text{ajuste de parámetros},$$
$$\text{validation} \rightarrow \text{selección de hiperparámetros},$$
$$\text{test} \rightarrow \text{estimación final}.$$

13.3. ¿Qué significa “comparación justa”?

Cambiar sólo una variable por vez. Si estamos comparando optimizadores, no deberíamos cambiar simultáneamente dimensión latente, dropout y número de épocas.

Para pensar: Adam puede converger más rápido en las primeras épocas. ¿Eso significa necesariamente que será mejor después de muchísimas épocas? No. La comparación depende del presupuesto de entrenamiento y de los hiperparámetros.

14. Ejercicio 14: Evaluación final sobre test

```
test_loss = evaluate(
    model,
    test_loader,
    loss_fn,
    device
)

print(f"Test MSE: {test_loss:.5f}")
```

En esta etapa la configuración ya fue elegida usando *validation*. *Test* responde ahora a una pregunta distinta:

> ¿Qué error de reconstrucción obtiene el modelo elegido sobre ejemplos que no participaron ni del aprendizaje ni de la comparación de hiperparámetros?

14.1. Qué no debemos hacer después

No deberíamos observar test MSE, cambiar el modelo para mejorarlo, volver a observar *test*, cambiar de nuevo y repetir muchas veces. Eso reutilizaría *test* como herramienta de ajuste.

15. Ejercicio 15: Visualizar reconstrucciones sobre test

```
@torch.no_grad()
def show_reconstructions(model, data_loader, device, n=6):
    model.eval()

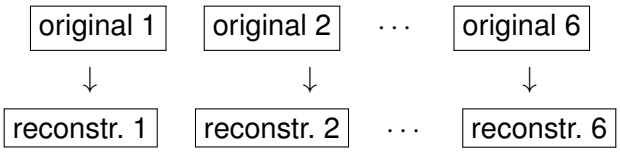
    images, _ = next(iter(data_loader))
    images = images[:n].to(device)

    reconstructions = model(images)

    images = images.cpu()
    reconstructions = reconstructions.cpu()

    fig, axes = plt.subplots(2, n, figsize=(2 * n, 4))
    ...
```

La comparación visual debe hacerse columna por columna:



15.1. Qué observar

- forma global de la prenda;
- bordes y contornos;
- pérdida de detalles finos;
- suavizado de regiones;
- diferencias entre clases visualmente simples y complejas.

Un *bottleneck* pequeño suele favorecer reconstrucciones más suaves: la red retiene regularidades globales y puede descartar detalles.

16. Ejercicio 16: Observar el espacio latente

```
images, _ = next(iter(test_loader))
images = images.to(device)

model.eval()

with torch.no_grad():
    latent_codes = model.encoder(images)

print("Imagenes:", images.shape)
print("Codigos latentes:", latent_codes.shape)
```

Con mini-batch size 100 y dimensión latente 64:

$$(100, 1, 28, 28) \to (100, 64).$$

Cada fila del tensor `latent_codes` es la representación de una imagen.

16.1. ¿Qué contiene el código latente?

No podemos afirmar que una componente represente necesariamente “manga”, otra “calzado” y otra “color”. La red no fue entrenada con una restricción de interpretabilidad. El código es simplemente una representación interna que resulta útil para reconstruir.

16.2. ReLU y signo de las componentes

Como la última activación efectiva del *encoder*, en **modo evaluación**, es ReLU seguida de **Dropout inactivo**, las componentes del código latente son no negativas.

16.3. ¿Por qué Dropout no altera estos códigos en esta celda?

Porque antes se ejecuta

```
model.eval()
```

y en ese modo `Dropout` se comporta como identidad.

17. Ejercicio 17: Preguntas conceptuales - soluciones completas

17.1. ¿Por qué las etiquetas de Fashion-MNIST no son necesarias?

Respuesta: Porque el objetivo del autoencoder no es predecir la clase y, sino reconstruir la entrada x. La función de pérdida *“loss”* compara $\hat{x}$ contra x. Las etiquetas sólo se utilizan en esta guía para visualizar y reconocer ejemplos.

17.2. ¿Qué representa el código latente?

Respuesta: Es la representación interna producida por el encoder. Con dimensión latente 64, cada imagen de 784 píxeles se representa mediante 64 activaciones. Es una representación aprendida específicamente para permitir que el *decoder* reconstruya la imagen.

17.3. ¿Qué significa el bottleneck?

Respuesta: Es una capa intermedia de dimensión menor que la entrada. Obliga a la red a pasar la información a través de una **representación más compacta**. En esta guía el *bottleneck* reduce 784 componentes a 64.

17.4. ¿Por qué usamos MSELoss?

Respuesta: Porque necesitamos medir una diferencia numérica entre la imagen original y la reconstrucción. MSE promedia los errores cuadráticos de los píxeles. Un valor menor indica que, en promedio, los píxeles reconstruidos están más cerca de los originales.

17.5. ¿Qué hace `optimizer.zero_grad()`?

Respuesta: Borra los gradientes almacenados de pasos anteriores. PyTorch acumula gradientes por defecto; si no los borráramos, `backward()` sumaría el gradiente nuevo al anterior.

17.6. ¿Qué hace `loss.backward()`?

Respuesta: Calcula mediante diferenciación automática los gradientes de la loss respecto de todos los parámetros entrenables que participaron en el forward. No modifica todavía los parámetros.

17.7. ¿Qué hace `optimizer.step()`?

Respuesta: Utiliza los gradientes ya calculados para actualizar los parámetros según la regla del optimizador elegido. Es la operación que efectivamente cambia pesos y biases.

17.8. ¿Por qué volvemos a evaluar train después de cada época?

Respuesta: Porque durante la época los pesos van cambiando mini-batch a mini-batch. Al terminar, evaluar nuevamente train permite calcular una loss asociada a un único estado fijo del modelo: los pesos finales de esa época. Así se puede comparar coherentemente con validation, evaluada con esos mismos pesos.

17.9. ¿Qué diferencia hay entre `model.train()` y `model.eval()`?

Respuesta: No activan ni desactivan autograd. Cambian el comportamiento de ciertos módulos. En esta red, `model.train()` activa Dropout y `model.eval()` lo desactiva. Para desactivar gradientes durante evaluación utilizamos además `torch.no_grad()`.

17.10. ¿Qué hace `Dropout(p)` durante `model.train()`?

Respuesta: Anula aleatoriamente cada activación de la capa donde está aplicado con probabilidad p. Las activaciones que sobreviven se reescalan para mantener aproximadamente su valor esperado. Esto introduce ruido de regularización durante el entrenamiento.

17.11. ¿Qué ocurre con dropout durante `model.eval()`?

Respuesta: Se desactiva. La capa Dropout se comporta como identidad. De esta manera la inferencia es determinista para una entrada fija y utiliza todas las activaciones.

17.12. ¿Qué significa elegir `DROPOUT_P=0.0`?

Respuesta: Que la probabilidad de anular una activación es cero. En la práctica, Dropout queda desactivado también durante entrenamiento. Es útil para comparar el autoencoder base con una versión regularizada.

17.13. ¿Por qué no colocamos dropout en la capa final?

Respuesta: Porque la capa final representa directamente los píxeles reconstruidos. Anular aleatoriamente componentes de esa salida introduciría ruido en los valores que estamos comparando con la imagen objetivo. En esta guía Dropout actúa sobre la representación interna.

17.14. ¿Por qué SGD y Adam usan learning rates diferentes?

Respuesta: Porque los algoritmos transforman los gradientes de manera distinta. Adam adapta la magnitud de sus actualizaciones mediante estadísticas de los gradientes, mientras que SGD básico usa directamente el gradiente multiplicado por el learning rate. Por eso sus escalas habituales de η no tienen que coincidir.

18. Ejercicio 18: Experimentos sugeridos - guía de solución

La notebook propone modificar una sola variable por vez. Esta regla es esencial para que el efecto observado pueda atribuirse al hiperparámetro estudiado.

18.1. Experimento A: dimensión latente

Probar, por ejemplo:

```
LATENT_DIM = 16
LATENT_DIM = 32
LATENT_DIM = 64
LATENT_DIM = 128
```

18.1.1. Efectos esperados

Al disminuir la dimensión latente:

- aumenta la compresión de representación;
- disminuye la capacidad del cuello de botella;
- puede aumentar el MSE;
- pueden perderse detalles finos.

Al aumentarla:

- el modelo dispone de más capacidad;
- puede reconstruir mejor;
- el *bottleneck* es menos restrictivo;
- si la dimensión se vuelve demasiado grande, la tarea de reconstrucción puede hacerse demasiado fácil y la representación ser menos compacta.

18.1.2. Cómo resolver el experimento correctamente

Mantener fijos:

- optimizador;
- learning rate;
- dropout;
- mini-batch size;
- número de épocas;
- split train/validation.

Comparar validation MSE y reconstrucciones visuales.

18.2. Experimento B: dropout

Probar:

```
DROPOUT_P = 0.0
DROPOUT_P = 0.2
DROPOUT_P = 0.5
```

18.2.1. Interpretación esperada

- $p = 0$: modelo sin ruido de dropout;
- $p = 0{,}2$: regularización moderada;
- $p = 0{,}5$: perturbación mucho más fuerte del código latente.

Un dropout demasiado alto puede dificultar la reconstrucción, especialmente porque el bottleneck ya constituye una restricción fuerte.

> **Observación:** No debe prometerse que Dropout mejorará validation MSE. Es un hiperparámetro que puede ayudar, no una garantía. En autoencoders simples incluso puede empeorar la reconstrucción si la regularización es excesiva.

18.3. Experimento C: número de épocas

Probar 5, 10 y 20 épocas manteniendo el resto fijo.

Con pocas épocas puede haber subentrenamiento. Con más épocas la loss puede seguir bajando, pero aumenta el costo computacional y eventualmente puede aparecer una separación entre train y validation.

18.4. Experimento D: SGD versus Adam

Usar:

Optimizador	Learning rate base	Observación
SGD	0.01	actualización directa con gradiente
Adam	0.001	actualización adaptativa

La comparación debe realizarse con validation. Test se consulta una vez elegida la configuración.

19. Errores frecuentes y cómo explicarlos

19.1. Usar las etiquetas en la loss

Error conceptual:

```
loss = loss_fn(reconstructions, labels)
```

No tiene sentido: `reconstructions` tiene forma de imagen y `labels` contiene enteros de clase.

Correcto:

```
loss = loss_fn(reconstructions, images)
```

19.2. Olvidar mover las imágenes al dispositivo

Si el modelo está en CUDA y los datos en CPU, aparecerá un error de dispositivos incompatibles.

19.3. Olvidar `model.eval()` durante evaluación

Con dropout activo, la loss de validation contendría ruido adicional y no representaría el comportamiento de inferencia del modelo.

19.4. Creer que `no_grad()` desactiva dropout

No. `no_grad()` afecta autograd. `eval()` afecta dropout.

19.5. Usar test para seleccionar el optimizador

Eso contamina la evaluación final. El **optimizador es un hiperparámetro** y debe elegirse con *validation*.

19.6. Confundir vector latente con etiqueta

La **representación latente** no es una clase ni una probabilidad. Es una **representación continua aprendida**.

19.7. Interpretar dimensión latente como bytes comprimidos

La dimensión cuenta componentes matemáticas, no tamaño real de almacenamiento en un archivo.

20. Resumen conceptual

Concepto	Significado en esta guía
Entrada	imagen x de Fashion-MNIST
Objetivo	la misma imagen x
Encoder	transforma 784 valores en 64
Código latente	representación interna z
Bottleneck	restricción de dimensión
Decoder	reconstruye 784 valores
Sigmoid	mantiene salida en $(0, 1)$
MSE	mide error de reconstrucción
Dropout	regulariza el código durante train
SGD/Adam	actualizan parámetros
Validation	permite comparar configuraciones
Test	evaluación final

La ecuación central de toda la guía puede resumirse en

$$\hat{\boldsymbol{x}} = f_{\mathrm{dec},\theta}\left(f_{\mathrm{enc},\theta}(\boldsymbol{x})\right),$$

con entrenamiento orientado a minimizar el Error Cuadrático Medio (MSE)

$$\mathrm{MSE}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \frac{1}{D}\sum_{j=1}^{D}(x_j - \hat{x}_j)^2.$$

En nuestro caso, para 784 píxeles:

$$D = 784.$$

A. Código completo consolidado

Listing 1: Solución completa consolidada de la notebook.

```
# -*- coding: utf-8 -*-
# Solucion completa consolidada de la notebook
# neuronales_2026_autoencoder_fashion_mnist_v6_resuelta.ipynb

# Ejercicio 1) Preparar el entorno | 1.1) Importar librerías
import random

import matplotlib.pyplot as plt
import numpy as np
import torch

from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

# 1.2) Reproducibilidad y dispositivo
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("PyTorch:", torch.__version__)
print("Dispositivo:", device)

# Ejercicio 2) Cargar y preparar Fashion-MNIST | 2.1) Transformación
transform = transforms.ToTensor()

# 2.2) Train, validation y test
full_train_set = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=transform
)

test_set = datasets.FashionMNIST(
    root="data",
    train=False,
```

```
    download=True,
    transform=transform
)

split_generator = torch.Generator().manual_seed(SEED)

train_set, valid_set = random_split(
    full_train_set,
    [50_000, 10_000],
    generator=split_generator
)

print("-------------------")
print("Train:", len(train_set))
print("Validation:", len(valid_set))
print("Test:", len(test_set))

# 2.3) Inspeccionar un ejemplo
image, label = train_set[0]

print("Shape:", image.shape)
print("Etiqueta:", label)
print("Mínimo:", image.min().item())
print("Máximo:", image.max().item())

# 2.4) Mosaico 3×3 de imágenes con sus etiquetas
class_names = {
    0: "T-shirt/top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle boot",
}

display_generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(
    len(train_set),
    generator=display_generator
)[:9].tolist()

fig, axes = plt.subplots(3, 3, figsize=(7, 7))

for ax, idx in zip(axes.flat, indices):
    image, label = train_set[idx]

    ax.imshow(image.squeeze(), cmap="gray")
    ax.set_title(class_names[label])
    ax.axis("off")

plt.tight_layout()
plt.show()

# Ejercicio 3) Crear los ‘DataLoader‘s
BATCH_SIZE = 100

train_loader = DataLoader(
    train_set,
```

```
    batch_size=BATCH_SIZE,
    shuffle=True
)

valid_loader = DataLoader(
    valid_set,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_set,
    batch_size=BATCH_SIZE,
    shuffle=False
)

images, labels = next(iter(train_loader))

print("images.shape:", images.shape)
print("labels.shape:", labels.shape)

# Ejercicio 4) Comprender el autoencoder
# Ejercicio 5) Definir el autoencoder | Dropout opcional
class Autoencoder(nn.Module):
    def __init__(self, latent_dim=64, p_dropout=0.0):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, latent_dim),
            nn.ReLU(),
            nn.Dropout(p_dropout)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 28 * 28),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        x_hat = self.decoder(z)

        # Recuperamos la forma original de las imágenes.
        x_hat = x_hat.view(x.size(0), 1, 28, 28)

        return x_hat

# 5.1) Comprobar dimensiones
LATENT_DIM = 64
DROPOUT_P = 0.2   # Usar 0.0 para desactivar dropout

torch.manual_seed(SEED)

model_example = Autoencoder(
    latent_dim=LATENT_DIM,
    p_dropout=DROPOUT_P
)

# Para esta comprobación queremos un comportamiento determinista.
model_example.eval()
```

```
with torch.no_grad():
    reconstructions = model_example(images)
    latent_codes = model_example.encoder(images)

print("Entrada:", images.shape)
print("Código latente:", latent_codes.shape)
print("Reconstrucción:", reconstructions.shape)
print("Dropout p:", DROPOUT_P)

assert reconstructions.shape == images.shape
assert latent_codes.shape == (images.shape[0], LATENT_DIM)

# Ejercicio 6) Función de pérdida
loss_fn = nn.MSELoss()

# Ejercicio 7) Elegir el optimizador
OPTIMIZER_NAME = "Adam"   # Opciones: "SGD" o "Adam"

SGD_LR = 0.01
ADAM_LR = 0.001

# Ejercicio 8) Implementar una época de entrenamiento
def train_one_epoch(model, data_loader, loss_fn, optimizer, device):
    model.train()

    for images, _ in data_loader:
        images = images.to(device)

        optimizer.zero_grad()

        reconstructions = model(images)
        loss = loss_fn(reconstructions, images)

        loss.backward()
        optimizer.step()

# Ejercicio 9) Implementar la evaluación
@torch.no_grad()
def evaluate(model, data_loader, loss_fn, device):
    model.eval()

    total_loss = 0.0
    total_examples = 0

    for images, _ in data_loader:
        images = images.to(device)

        reconstructions = model(images)
        loss = loss_fn(reconstructions, images)

        batch_size = images.size(0)

        total_loss += loss.item() * batch_size
        total_examples += batch_size

    mean_loss = total_loss / total_examples

    return mean_loss

# Ejercicio 10) Crear el modelo y el optimizador
torch.manual_seed(SEED)
```

```
model = Autoencoder(
    latent_dim=LATENT_DIM,
    p_dropout=DROPOUT_P
).to(device)

if OPTIMIZER_NAME == "SGD":
    optimizer = torch.optim.SGD(
        model.parameters(),
        lr=SGD_LR
    )

elif OPTIMIZER_NAME == "Adam":
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=ADAM_LR
    )

else:
    raise ValueError("OPTIMIZER_NAME debe ser 'SGD' o 'Adam'.")

print("Optimizador:", OPTIMIZER_NAME)
print("Dropout p:", DROPOUT_P)

# 10.1) Reconstrucciones antes del entrenamiento
@torch.no_grad()
def show_untrained_reconstructions(model, data_loader, device, n=6):
    model.eval()

    images, labels = next(iter(data_loader))

    images = images[:n].to(device)
    labels = labels[:n]

    reconstructions = model(images)

    images = images.cpu()
    reconstructions = reconstructions.cpu()

    fig, axes = plt.subplots(
        2,
        n,
        figsize=(2 * n, 4)
    )

    for i in range(n):
        axes[0, i].imshow(
            images[i].squeeze(),
            cmap="gray"
        )
        axes[0, i].set_title(
            class_names[labels[i].item()],
            fontsize=9
        )
        axes[0, i].axis("off")

        axes[1, i].imshow(
            reconstructions[i].squeeze(),
            cmap="gray"
        )
        axes[1, i].axis("off")

    axes[0, 0].set_ylabel("Original")
```

```
    axes[1, 0].set_ylabel("Sin entrenar")

    plt.tight_layout()
    plt.show()


show_untrained_reconstructions(
    model,
    valid_loader,
    device
)

# Ejercicio 11) Entrenar el modelo
EPOCHS = 10

history = {
    "train_loss": [],
    "valid_loss": []
}

for epoch in range(1, EPOCHS + 1):
    train_one_epoch(
        model,
        train_loader,
        loss_fn,
        optimizer,
        device
    )

    train_loss = evaluate(
        model,
        train_loader,
        loss_fn,
        device
    )

    valid_loss = evaluate(
        model,
        valid_loader,
        loss_fn,
        device
    )

    history["train_loss"].append(train_loss)
    history["valid_loss"].append(valid_loss)

    print(
        f"Época {epoch:02d}/{EPOCHS} | "
        f"train MSE={train_loss:.5f} | "
        f"valid MSE={valid_loss:.5f}"
    )

# Ejercicio 12) Graficar las pérdidas
epochs = range(1, EPOCHS + 1)

plt.figure(figsize=(7, 4))
plt.plot(epochs, history["train_loss"], marker="o", label="train")
plt.plot(epochs, history["valid_loss"], marker="o", label="validation")
plt.xlabel("Época")
plt.ylabel("MSE")
plt.title(f"Autoencoder - {OPTIMIZER_NAME}")
plt.legend()
```

```
plt.grid(alpha=0.3)
plt.show()

# Ejercicio 13) Comparar SGD y Adam utilizando validation
# Ejercicio 14) Evaluar la configuración seleccionada sobre test
test_loss = evaluate(
    model,
    test_loader,
    loss_fn,
    device
)

print(f"Test MSE: {test_loss:.5f}")

# Ejercicio 15) Visualizar reconstrucciones sobre test
@torch.no_grad()
def show_reconstructions(model, data_loader, device, n=6):
    model.eval()

    images, _ = next(iter(data_loader))
    images = images[:n].to(device)

    reconstructions = model(images)

    images = images.cpu()
    reconstructions = reconstructions.cpu()

    fig, axes = plt.subplots(2, n, figsize=(2 * n, 4))

    for i in range(n):
        axes[0, i].imshow(images[i].squeeze(), cmap="gray")
        axes[0, i].axis("off")

        axes[1, i].imshow(reconstructions[i].squeeze(), cmap="gray")
        axes[1, i].axis("off")

    axes[0, 0].set_ylabel("Original")
    axes[1, 0].set_ylabel("Reconstrucción")

    plt.tight_layout()
    plt.show()


show_reconstructions(model, test_loader, device)

# Ejercicio 16) Observar el espacio latente
images, _ = next(iter(test_loader))
images = images.to(device)

model.eval()

with torch.no_grad():
    latent_codes = model.encoder(images)

print("Imágenes:", images.shape)
print("Códigos latentes:", latent_codes.shape)

# Ejercicio 17) Preguntas conceptuales
# Ejercicio 18) Experimentos sugeridos | 18.1) Dimensión latente | 18.2) Dropout | 18.3) Nú
    mero de épocas | 18.4) Optimizador
```

B. Referencias mínimas

- PyTorch: https://pytorch.org/
- Tutoriales básicos de PyTorch: https://pytorch.org/tutorials/beginner/basics/intro.html
- Fashion-MNIST: https://github.com/zalandoresearch/fashion-mnist
- Documentación de `torch.nn`: https://pytorch.org/docs/stable/nn.html